Kurzanleitung et-control

Helmut Neemann

13. November 2026

Inhaltsverzeichnis

1 Allgemeines **2**
1.1 Anweisungen . . . 3
1.2 Literale . . . 3
1.3 Spezielle Ausdrücke . . . 4
1.4 Anzeige mehrerer Werte . . . 5

2 Abbildungen **5**
2.1 Weitere Abbildungsinhalte . . . 7
2.2 Abbildungen als Werte . . . 7

3 Regelungstechnik **8**
3.1 komplexe Zahlen . . . 8
3.1.1 Methoden auf komplexen Zahlen . . . 8
3.2 Polynome . . . 8
3.2.1 Methoden auf Polynomen . . . 9
3.3 lineare Systeme . . . 9
3.3.1 Methoden auf linearen Systemen . . . 10
3.3.2 Die Funktion `pid` . . . 11
3.4 Ortskurven . . . 11
3.5 Bode-Diagramme . . . 12
3.6 Wurzelortskurven . . . 13
3.6.1 Evans-Plot . . . 13
3.6.2 Allgemeine Wurzelortskurve . . . 14
3.7 Simulation linearer Systeme . . . 15

4 Interaktivität **17**

5 Listen **18**

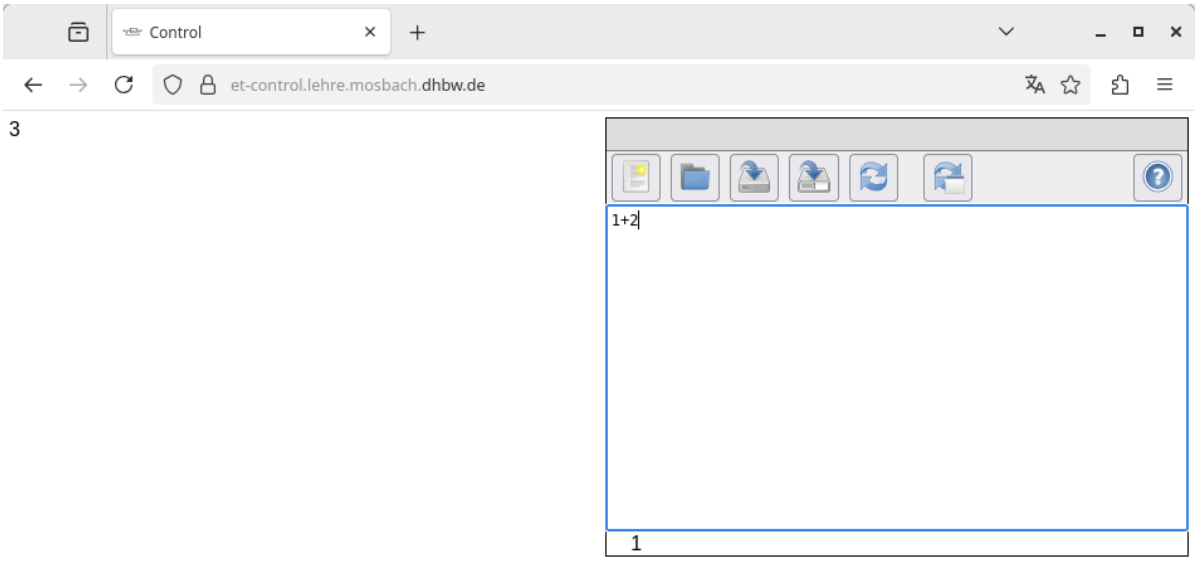


Abbildung 1: Browserfenster

1 Allgemeines

Bei et-control handelt es sich um einen WEB-Service, der genutzt werden kann, um Berechnungen aller Art durchzuführen und Abbildungen von Funktionsgraphen o.ä. zu erzeugen. Zudem gibt es einige Funktionen, die für das Verständnis der klassischen Regelungstechnik besonders hilfreich sind.

Da es sich um einen WEB-Service handelt, ist keinerlei Installation auf einem Endgerät erforderlich. Genutzt werden kann das Tool im Browser sowohl auf Tablets als auch auf einem PC. Im Prinzip ist es auch auf einem Smartphone lauffähig, jedoch ist der Bildschirm für eine angenehme Benutzung zu klein. Erreichbar ist die WEB-Seite unter der URL `https://et-control.lehre.mosbach.dhbw.de/`.

Die prinzipielle Funktionsweise ist einfach: Der Benutzer gibt einen Ausdruck ein, dieser Ausdruck wird ausgewertet und das Ergebnis wird angezeigt. Das Ergebnis der Auswertung eines Ausdruckes kann eine einfache Zahl sein, es kann jedoch auch eine Abbildung oder eine Liste von Abbildungen sein, die dann im Browser angezeigt werden.

Das Browser-Fenster ist dabei in zwei Bereiche aufgeteilt: Auf der rechten Seite bzw. unten im Porträt-Modus ist ein Eingabefenster, welches die Bearbeitung eines Ausdruckes erlaubt, und auf der linken Seite bzw. oben im Porträt-Modus die Ausgabe. Die Abbildung 1 zeigt das Browser-Fenster.

Bei einem Klick auf das Fragezeichen kann eine Liste von Beispielen geöffnet werden, die als erster Einstieg in die Erstellung von Ausdrücken gedacht sind. Wenn der Ausdruck verändert wird, kann mit einem Klick auf das Aktualisieren-Icon die Ausgabe aktualisiert werden.

1.1 Anweisungen

Es können in einem Ausdruck lediglich zwei Anweisungen benutzt werden, die jeweils einem Ausdruck vorangestellt werden können. Die erste ist die `let`-Anweisung. Mit ihr kann eine Konstante angelegt werden. Die Anweisung endet mit einem Semikolon. Danach kommt der Ausdruck der berechnet werden soll. Es dürfen mehrere `let` Anweisungen benutzt werden.

```
let a = 1+2;
let b = 2^2;

a+b
```

Die zweite Anweisung ist die `func` Anweisung. Sie dient der Deklaration von Funktionen. Auch sie wird mit einem Semikolon abgeschlossen.

```
func f(x)
  x+2;

let a = 1+2;

f(a)
```

1.2 Literale

Es können einige wenige Literale verwendet werden, die im folgenden beschrieben werden:

Zeichenketten Mit Anführungszeichen können Zeichenketten erzeugt werden:

```
let a = "Test";

a
```

Listen Mit eckigen Klammen können Listen angelegt werden:

```
[1,2,3]
```

zugegriffen wird auf die Elemente der Liste über einen Index:

```
let l = [1,2,3];

l[0]
```

Listen sind ein sehr häufig verwendeter Typ und es gibt eine ganze Reihe von Methoden, die auf Listen aufgerufen werden können. Mehr dazu im Kapitel 5.

Maps Mit geschweiften Klammen können Maps angelegt werden:

```
let m = {a:1, b:2};

m.a
```

Funktionen Mit dem Pfeiloperator `->` können anonyme Funktionen deklariert werden:

```
let f = x->x^2;

f(2)
```

oder

```
func call(f)
  f(2);

call(x->x^2)
```

Wenn eine anonyme Funktion mehr als ein Argument haben soll, sind die Argumente in Klammern zu setzen:

```
let f = (x,y) -> sqrt(x^2+y^2);

f(3,4)
```

Tatsächlich ist die weiter oben beschrieben Anweisung `func` nur eine etwas andere Schreibweise der `let`-Anweisung, denn intern entspricht die Anweisung

```
func f(x)
  x^2;
```

dem Ausdruck

```
let f = x->x^2;
```

Funktionen haben stets genau einen Rückgabewert. Sollen mehrere Werte zurückgegeben werden, muss dies als Liste oder Map geschehen:

```
func f(x)
  [x^2,x^3];
```

oder

```
func f(x)
  {a:x^2, b:x^3};
```

1.3 Spezielle Ausdrücke

Es gibt drei spezielle Ausdrücke, die vergleichsweise ungewöhnlich sind:

if Eine Bedingung kann mit dem `if` Ausdruck überprüft werden. Es muss immer einen `else` Teil geben, da es sich um einen Ausdruck handelt der immer einen Rückgabewert haben muss:

```
let b = 3;
let a = if b>1 then b+1 else b-1;

a
```

switch Es kann ein `switch` Ausdruck verwendet werden der ebenfalls ein `default` haben muss, um sicherzustellen, dass stets ein Wert zurückgegeben wird:

```
let x = 1;
let b = 2;
let a = switch b
          case 1: sin(x)
          case 2: cos(x)
          default x;

a
```

try Der Ausdruck `try` dient der Fehlerbehandlung. Er wird genutzt, um auf Fehler zu reagieren. Sollte bei der Berechnung ein Fehler auftreten, kann ein alternativer Ausdruck ausgewertet werden.

```
let m = {a:1, b:2};

let e = try m.c catch 3;

e
```

Der Ausdruck `m.c` löst einen Fehler aus, da es in der Map `m` keinen Eintrag `c` gibt. Dieser Fehler wird durch das `try` eingefangen und anstelle einer Laufzeitfehlermeldung wird der `catch`-Ausdruck ausgewertet und eine 3 zurückgegeben.

1.4 Anzeige mehrerer Werte

Der Ausdruck, der ausgewertet wird, hat immer nur einen Rückgabewert. Sollen mehrere Werte angezeigt werden, kann jedoch eine Liste oder eine Map zurückgegeben werden. Sollen z.B. mehrere Funktionsresultate angezeigt werden:

```
func f(x)
  x^2;

[
 f(1),
 f(3),
 f(7)
]
```

2 Abbildungen

Das Ergebnis der Auswertung eines Ausdruckes kann auch eine Abbildung sein. Diese Abbildung wird bei der Ausgabe dann vom Browser angezeigt. Abbildungen sind ganz normale Elemente vergleichbar mit einer einfachen Zahl oder auch einer Liste. So kann

ein Ausdruck auch eine Liste von Abbildungen zurückgeben, die dann auch als Liste von Abbildungen angezeigt wird.

Erzeugt wird eine Abbildung mit der `plot(content...)`-Funktion. An sie werden als Argumente ein oder mehrere Abbildungsinhalte übergeben und der Rückgabewert ist die entsprechende Abbildung. Soll z.B. eine Sinusfunktion dargestellt werden, so sind zwei Schritte erforderlich: Es muss ein Abbildungsinhalt aus der Sinusfunktion erzeugt werden, also der Graph dieser Funktion und dieser muss dann an die `plot`-Funktion übergeben werden, um aus dem Graphen eine Abbildung zu erzeugen:

```
func f(x)
  sin(x);

plot(
  f.graph()
)
```

Die `graph`-Methode erzeugt einen darstellbaren Graphen aus der Funktion auf der diese Methode aufgerufen wurde. Sie kann auf jeder Funktion aufgerufen werden. Vereinfacht ließe sich also auch schreiben:

```
plot(
  (x->sin(x)).graph()
)
```

Werden keine Grenzen angegeben, wird der Bereich $-1 < x < 1$ dargestellt. Diese Grenzen können mit der Methode `xBounds(xMin, xMax)` verändert werden. Diese Methode kann auf einer Abbildung aufgerufen werden und der Rückgabewert ist eine neue Abbildung mit den veränderten Grenzen:

```
plot(
  (x->sin(x)).graph()
).xBounds(0, 2*pi)
```

Es können auch mehrere Graphen dargestellt werden:

```
plot(
  (x->sin(x)).graph(),
  (x->cos(x)).graph()
).xBounds(0, 2*pi)
```

Auch auf Graphen können verschiedene Methoden aufgerufen werden, so kann z.B. eine Farbe und eine Titel gesetzt werden indem auf dem Graphen die Methode `line` Aufgerufen wird: Sie liefert einen neuen Graphen mit geänderter Farbe und entsprechendem Titel zurück.

```
plot(
  (x->sin(x)).graph().line(black, "sin(x)"),
  (x->cos(x)).graph().line(red, "cos(x)")
).xBounds(0, 2*pi)
```

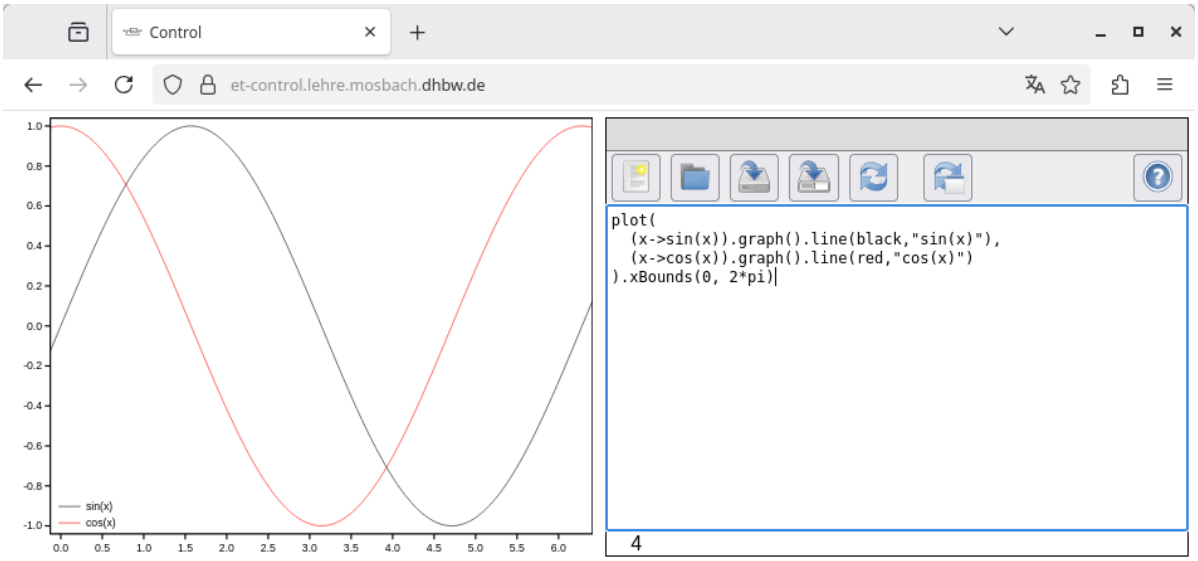


Abbildung 2: Zwei Funktionen

Das Ergebnis dieses Ausdruckes ist in Abbildung 2 gezeigt.

Soll eine Abbildung exportiert werden, z.B. um sie in ein LaTeX-Dokument einzubilden, kann auf der Abbildung die Methode `svg` aufgerufen werden. Zurückgegeben wird dann ein Link auf die SVG-Datei, über welchen die Datei gespeichert werden kann:

```
plot(
  (x->sin(x)).graph().line(black,"sin(x)"),
  (x->cos(x)).graph().line(red,"cos(x)")
).xBounds(0, 2*pi)
 .svg("nameDerDatei")
```

2.1 Weitere Abbildungsinhalte

Im folgenden einige nützliche Abbildungsinhalte:

yConst(y) Ein konstantes y. Die Konstante wird gestrichelt gezeichnet.

xConst(x) Ein konstantes x. Die Konstante wird gestrichelt gezeichnet.

text(x,y,text) An der angegeben Position wird ein Text angezeigt.

arrow(x1,y1,x2,y2,text) Es wird ein beschrifteter Pfeil angezeigt.

2.2 Abbildungen als Werte

Abbildungen sind ganz normale Werte und können auch genauso so behandelt werden. So lässt sich z.B. eine Liste mit mehreren Abbildungen erzeugen:

```
let p = plot( (x->sin(x)).graph() );

[p, p.xBounds(0,2*pi)]
```

Zunächst wird dem Bezeichner p eine Abbildung zugewiesen und dann wird eine Liste mit eben dieser Abbildung und einer weiteren Abbildung erzeugt, bei welcher die Grenzen verändert wurden.

3 Regelungstechnik

In der klassischen Regelungstechnik spielt das lineare System, welches beschrieben werden kann als Quotient zweier Polynome, eine wichtige Rolle. Dabei werden die Polynome für komplexe Zahlen ausgewertet. Daher gibt es neben den komplexen Zahlen auch das Polynom und das lineare System als eigene, native Datentypen.

3.1 komplexe Zahlen

Ein komplexe Zahl wird durch die Funktion `cmplx([re],[im])` erzeugt. Zudem gibt es die globale Konstante `j=cmplx(0,1)` als imaginäre Einheit.

Mit komplexen Zahlen kann ganz normal gerechnet werden. Alle Grundrechenarten werden unterstützt.

3.1.1 Methoden auf komplexen Zahlen

Im Folgenden die Methoden, die auf komplexen Zahlen definiert sind:

real() Gibt den Realteil der komplexen Zahl zurück.

imag() Gibt den Imaginärteil der komplexen Zahl zurück.

conj() Erzeugt die entsprechend konjugiert komplexe Zahl.

abs() Liefert den Betrag der komplexen Zahl.

phase() Liefert die Phase der komplexen Zahl im Bogenmaß im Bereich zwischen $-\pi$ und π. Negative Imaginärteile liefern einen negativen Winkel.

3.2 Polynome

Ein Polynom wird erzeugt mit der Funktion `poly(c0, c1, ...)`. Übergeben werden die stets reellen Koeffizienten des Polynoms und zurückgegeben wird das Polynom. So wird das Polynom $3x^2 + 2x + 1$ erzeugt durch die Anweisung

```
poly(1,2,3)
```

Polynome können addiert, subtrahiert und multipliziert werden.

```
let p = poly(1,2,3)*poly(2,1);
p
```

Das einfache Polynom x wird damit erzeugt durch `poly(0,1)`. Dieses Polynom ist als konstanter Wert mit dem Namen `s` angelegt und kann direkt verwendet werden. Damit lässt sich das obige Polynom $3x^2 + 2x + 1$ auch erzeugen durch die Anweisung:

```
let p = 3*s^2+2*s+1;
p
```

Polynome können wie Funktionen verwendet werden. So liefert

```
let p = 3*s^2+2*s+1;
p(2)
```

den Wert 17. Als Argument kann auch eine komplexe Zahl verwendet werden. So liefert

```
let p = 3*s^2+2*s+1;
p(1+j*2)
```

den Wert $-6 + 16i$

3.2.1 Methoden auf Polynomen

Im Folgenden einige nützliche Methoden, die auf Polynomen definiert sind:

derivative() Gibt die Ableitung des Polynoms zurück.

roots() Gibt die Wurzeln (Nullstellen) des Polynoms zurück. Komplexe Wurzeln werden ebenfalls berechnet. Berechnet werden die Wurzeln mittels des Newton-Verfahrens mit anschließender Polynomdivision bis die Ordnung des resultierenden Polynoms auf Zwei gefallen ist.

bode() Liefert das Bode-Diagramm des Polynoms als Abbildungsinhalt zurück. Weiteres zu Bode-Diagrammen siehe in Kap. 3.5.

3.3 lineare Systeme

Werden zwei Polynome durcheinander geteilt, ist das Ergebnis ein lineares System. Auch lineare Systeme können - genau wie Polynome - direkt addiert, subtrahiert, multipliziert und dividiert werden. So lässt sich das lineare System

$$G(s) = \frac{s+1}{3s^2+2s+1}$$

eingeben als

```
let g = poly(1,1)/poly(1,2,3);
g
```

oder als

```
let g = (s+1)/(3*s^2+2*s+1);
g
```

Wie Polynome können auch lineare Systeme wie Funktionen verwendet werden. So liefert

```
let g = (s+7)/(s^2+2*s+1);
g(1)
```

den Wert 2. Und auch hier kann das Argument eine komplexe Zahl sein.

3.3.1 Methoden auf linearen Systemen

Im Folgenden einige nützliche Methoden, die auf linearen Systemen definiert sind:

loop() Wird diese Methode auf einem linearen System aufgerufen, wird das lineare System des geschlossenen Kreises zurückgegeben. Wenn die Methode `loop` auf dem System $G_0(s) = Z(s)/N(s)$ aufgerufen wird, wobei mit $N(s)$ das Nenner- und mit $Z(s)$ das Zählerpolynom bezeichnet ist, wird

$$G(s) \quad = \quad \frac{G_0(s)}{1 + G_0(s)} = \frac{Z(s)}{N(s) + Z(s)}$$

zurückgegeben.

```
let g = (s+1)/(3*s^2+2*s+1);

g.loop()
```

pMargin() Diese Methode liefert eine Map zurück, in der unter `w0` die Frequenz mit $|G(j\omega_0)| = 1$ zurückgegeben wird und der Wert `pMargin` den Phasenrand enthält.

gMargin() Diese Methode liefert eine Map zurück, in der unter `w180` die Frequenz mit $k_m G(j\omega_{180}) = -1$ zurückgegeben wird und der Wert `gMargin` den Amplitudenrand k_m enthält. Der Amplitudenrand wird in dB angegeben.

numerator() Gibt das Zählerpolynom des linearen Systems zurück.

denominator() Gibt das Nennerpolynom des linearen Systems zurück.

derivative() Gibt die Ableitung des linearen Systems zurück.

poles() Gibt die Pole des linearen Systems zurück, also die Wurzeln des Nennerpolynoms.

zeros() Gibt die Nullstellen des linearen Systems zurück, also die Wurzeln des Zählerpolynoms.

reduce() Bestimmt die Wurzeln von Zähler und Nenner und kürzt gemeinsame Wurzeln. Zurückgegeben wird das vereinfachte lineare System.

nyquist() Liefert die Ortskurve des linearen Systems als Abbildungsinhalt zurück. Weiteres zu Ortskurven siehe in siehe Kap. 3.4.

bode() Liefert das Bode-Diagramm des linearen Systems als Abbildungsinhalt zurück. Weiteres zu Bode-Diagrammen siehe in Kap. 3.5.

evans(kMax) Liefert die Wurzelortskurve des linearen Systems als Abbildungsinhalt zurück. Weiteres zu Wurzelortskurven siehe Kap. 3.6.

simStep(tMax) Berechnung der Systemreaktion im Zeitbereich mit der Sprungfunktion als Eingangssignal. Weiteres zur Simulation linearer Systeme siehe in Kap. 3.7.

sim(func(t) u, tMax) Berechnung der Systemreaktion im Zeitbereich bei beliebigem Eingangssignal $u(t)$. Weiteres siehe Kap. 3.7.

3.3.2 Die Funktion `pid`

Es existiert die globale Funktion `pid`. Diese Funktion hat bis zu vier Argumente und vereinfacht die Erzeugung eines PID-Reglers.

```
let k_p = 0.1;
let T_I = 1;
let T_D = 0.1;
let T_P = 0.01;
let k = pid(k_p, T_I, T_D, T_P);
k
```

Dabei ist `k_p` der Proportionalteil, `T_I` die Integrationszeit und `T_D` die Vorhaltezeit des Differenzierers. Wenn auch noch das vierte Argument `T_P` angegeben wird, wird anstelle des idealen Differenzierers ein realer Differenzierer also ein Vorhalteglied verwendet. Die Zeitkonstante `T_P` fügt dem Differenzierer ein zusätzliches PT_1-Glied mit der entsprechenden Zeitkonstante hinzu, um den Regler realisierbar zu machen. Die letzten zwei Argumente sind optional. Zurückgegeben wird das lineare System des entsprechenden Reglers:

$$\begin{aligned} K(s) &= k_p(1 + \frac{1}{T_I s} + \frac{T_D s}{T_P s + 1}) \\ &= k_p \frac{T_I(T_D + T_P)s^2 + (T_I + T_P)s + 1}{T_I s(T_P s + 1)} \end{aligned}$$

3.4 Ortskurven

Auf dem linearen System ist die Methode `nyquist([neg [,wMax]])` definiert, welche den Graphen der Ortskurve des linearen Systems zurück gibt. Dieser Graph kann dann einer Abbildung hinzugefügt werden.

```
let g = (s+1)/(3*s^2+2*s+1);
plot(
  g.nyquist()
)
```

Die Methode `nyquist` hat ein optionales bool Argument. Wird dieses auf `true` gesetzt, dann wird nicht nur die komplette Ortskurve mit $-\infty < \omega < \infty$ angezeigt, sondern auch der kritische Punkt $-1 + j0$ hervorgehoben.

```
let g = (s+1)/(3*s^2+2*s+1);
plot(
  g.nyquist(true)
)
```

3.5 Bode-Diagramme

Auf Polynomen und linearen Systemen ist die Methode `bode([color [,titel]])` definiert. Sie liefert ein Bode-Diagramm als Abbildungsinhalt zurück welches einer Abbildung hinzugefügt werden kann.

Das Bode-Diagramm der obigen Übertragungsfunktion lässt sich also darstellen mit:

```
let g = (s+1)/(3*s^2+2*s+1);
plot(
  g.bode()
)
```

Der Methode `bode` kann eine Farbe und ein Titel mitgegeben werden.

```
let g = (s+1)/(3*s^2+2*s+1);
plot(
  g.bode(red, "G")
)
```

Standardmäßig wird das Bode-Diagramm in den Grenzen $0.01\,\text{rad/s} < \omega < 100\,\text{rad/s}$ dargestellt. Diese Grenzen können mit der Methode `xBounds(wMin,wMax)` gesetzt werden:

```
let g = (s+1)/(3*s^2+2*s+1);
plot(
  g.bode(red, "G")
).xBounds(0.1, 10)
```

Die Grenzen der Amplitude können mit `yBounds(aMin,aMax)` gesetzt werden und die Grenzen der Phase mit `ySecBounds(pMin,pMax)`, da die Phase auf der sekundären y-Achse eines Plots angezeigt wird.

Auch hier können die Graphen mehrerer Bode-Diagramme einer Abbildung hinzugefügt werden:

```
let g  = 3/((s+1)*(s+2));
let k  = 1.6*(1+1/(1.7*s));
let g0 = k*g;

plot(
  g.bode(green, "G"),
```

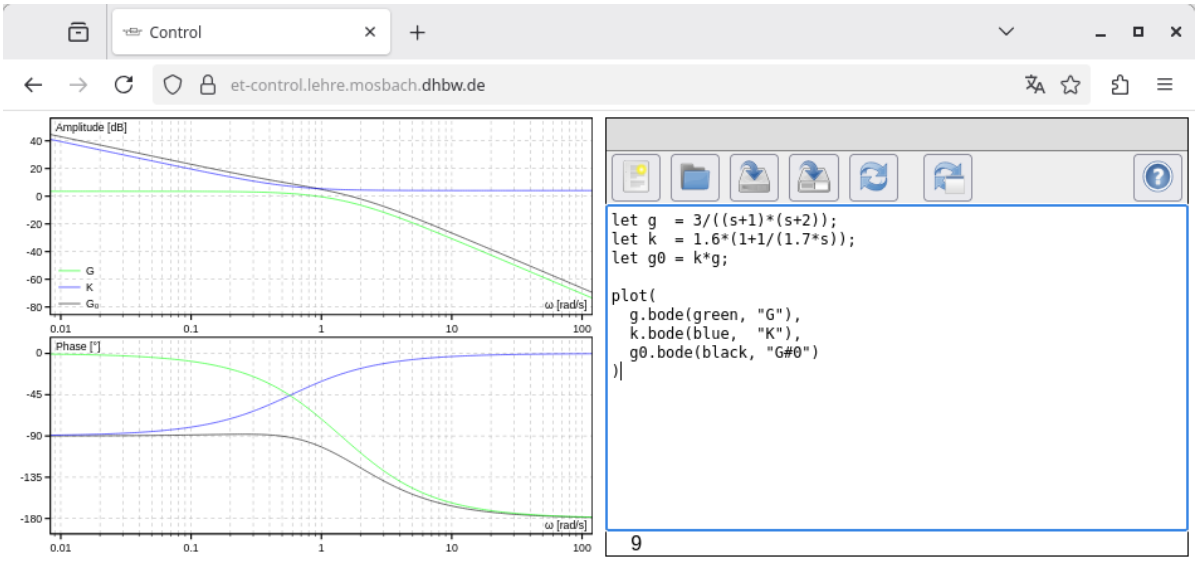


Abbildung 3: Bode-Diagramm

```
  k.bode(blue,  "K"),
  g0.bode(black, "G#0")
)
```

Das Ergebnis ist in Abbildung 3 gezeigt.

3.6 Wurzelortskurven

Die Wurzelortskurve stellt die Wurzelorte, also die Position der Wurzeln (Nullstellen) eines Polynoms bzw. die Lage der Pole eines linearen Systems in Abhängigkeit von einem Parameter in der komplexen Ebene dar. Wenn die Koeffizienten des Polynoms reell sind, ist die Wurzelortskurve spiegelsymmetrisch zu reellen Achse. Das ist hier stets der Fall, da die Koeffizienten der Polynome immer reell sind (siehe 3.2).

3.6.1 Evans-Plot

Die am meisten verwendete Variante einer Wurzelortskurve zeigt die Lage der Pole des geschlossenen Kreises eines linearen Systems mit der Verstärkung k der offenen Kette als Parameter. Diese Darstellung, nach seinem Entwickler Walter Richard Evans benannt, ist sehr Aussagekräftig und lässt sich zudem besonders leicht von Hand skizzieren.

Der Graph genau dieser Wurzelortskurve kann durch die Methode `evans(kMax)` erzeugt werden, die auf linearen Systemen definiert ist:

```
let g = (s+1)/(3*s^2+2*s+1);
plot(
```

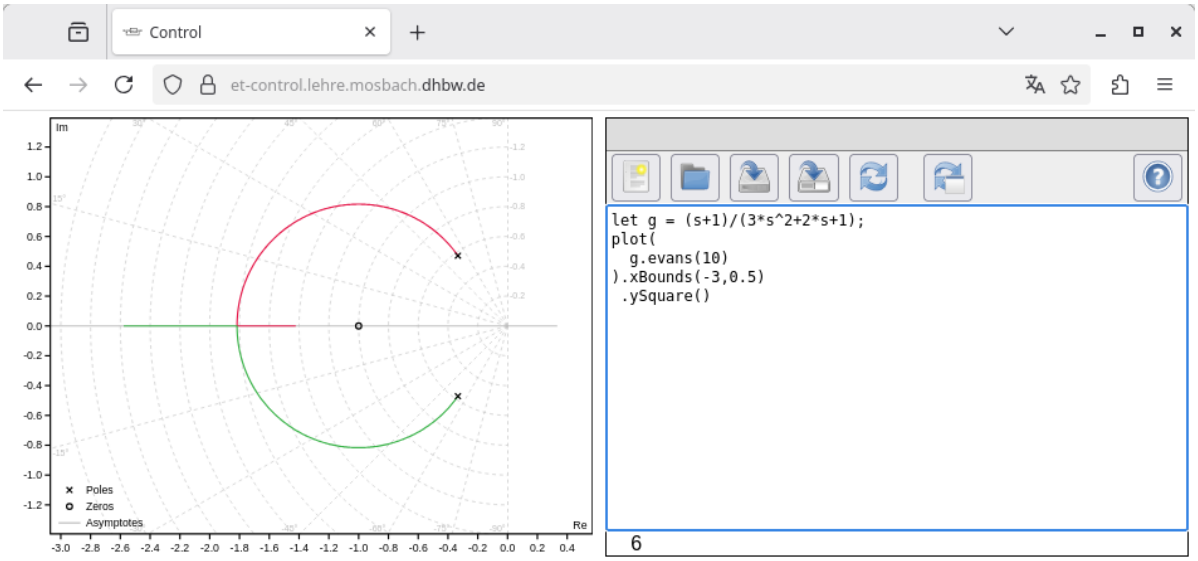


Abbildung 4: Wurzelortskurve in der üblichen Form: Lage der Pole des geschlossenen Kreise mit k als Parameter. Wobei mit k der Verstärkungsfaktor der offenen Kette bezeichnet ist.

```
  g.evans(10)
).xBounds(-3,0.5)
 .ySquare()
```

Das Argument der Methode `evans` gibt das maximale k an, bis zu welchem die Wurzelortskurve gezeichnet werden soll. Die Methode `ySquare()` ist definiert auf einer Abbildung und liefert eine neue Abbildung zurück, bei welcher die y-Achse so skaliert wird, dass x-Achse und y-Achse ein Seitenverhältnis von 1:1 besitzen. Abbildung 4 zeigt das Ergebnis.

Es ist zu beachten, dass das lineare System zunächst gekürzt wird. Gemeinsame Wurzeln von Zähler und Nenner tauchen also nicht als Pole in der Abbildung auf.

3.6.2 Allgemeine Wurzelortskurve

Soll eine Wurzelortskurve dargestellt werden, bei der nicht das k der offenen Kette der Parameter ist, dann kann die Methode `rootLocus(pMin, pMax, parName)` verwendet werden.

Die Funktion auf welcher diese Methode aufgerufen wird, muss entweder ein Polynom zurückgeben oder ein lineares System. Wenn ein Polynom zurückgegeben wird, wird die Ortskurve der Wurzeln dargestellt, wird ein lineares System zurückgegeben, wird die Ortskurve der Pole dargestellt. Zu beachten ist, dass im Falle eines lineares Systems eine gegebene offene Kette nicht automatisch geschlossen wird!

Als Beispiel soll die Wurzelortskurve des Systems

$$G(s) \quad = \quad \frac{7}{(s+1)(s+2)(s+2.5)}$$

incl. eines PID-Reglers angezeigt werden und zwar mit festem $k = 1$ und der Integrationszeit T_I als variablem Parameter.

Dazu muss zunächst eine Funktion deklariert werden, die das entsprechende System in Abhängigkeit von T_I erzeugt, und da wir die Pole des geschlossenen Kreises darstellen wollen, muss die offene Kette mit `loop` geschlossen werden:

```
let g = 7/((s+1)*(s+2)*(s+2.5));

func linSys(T_I)
  (pid(1, T_I)*g).loop();

plot(
  linSys.rootLocus(0.1, 10, "T_I")
)
```

Die Funktion `linSys` liefert das lineare System abhängig von T_I und die Methode `rootLocus` erzeugt den Graphen der Wurzelortskurve in den Grenzen $0.1\,\mathrm{s} < T_I < 10\,\mathrm{s}$.

Verkürzt hätte man auch schreiben können:

```
let g = 7/((s+1)*(s+2)*(s+2.5));

plot(
  ( T_I->(pid(1, T_I)*g).loop() ).rootLocus(0.1, 10, "T_I")
)
```

Auf diese Weise lassen sich beliebige Wurzelortskurven darstellen. Abbildung 5 zeigt das Ergebnis des obigen Ausdruckes.

3.7 Simulation linearer Systeme

Es kann die Reaktion eines linearen Systems im Zeitbereich numerische berechnet werden. Dazu sind zwei Methoden zur Simulation auf linearen System deklariert. Zunächst kann die Reaktion $h(t)$ eines linearen Systems im Zeitbereich auf einen Einheitssprung berechnet werden. Dazu dient die Methode `simStep([tMax])`. Das Argument `tMax` gibt an, bis zu welcher Zeit t_{Max} die Simulation durchgeführt werden soll. Die Rückgabe ist eine Liste mit (t,y)-Wertepaaren. Diese kann dann mit der Methode `graph` in einen Graphen für die Darstellung umgewandelt werden:

```
let g = 7/((s+1)*(s+2)*(s+2.5));
let k = pid(1,1.75);

plot(
  (k*g).loop().simStep(10).graph(),
```

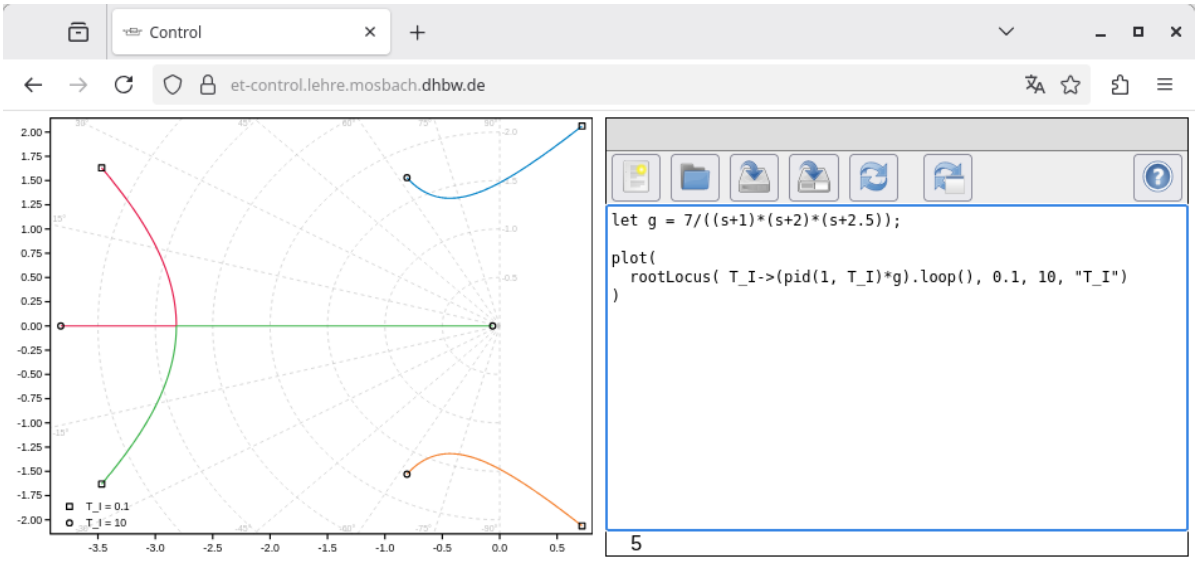


Abbildung 5: Wurzelortskurve des geschlossenen Kreises mit dem T_I eines PI-Reglers als Parameter.

```
  yConst(1)
).labels("t / s", "h(t)")
```

die Funktion `yConst(1)` ergänzt eine Gerade mit der Steigung 0 und $y = 1$. Abbildung 6 zeigt das Ergebnis.

Soll die Reaktion eines linearen Systems im Zeitbereich bei beliebigem Eingangssignal simuliert werden, kann die Methode `sim` aufgerufen werden. Auch hier wird zunächst eine Liste mit (t,y)-Wertepaaren zurückgegeben, welche dann mit der Methode `graph` in einen Graphen für die Darstellung umgewandelt werden kann.

Die Methode `sim` hat zwei Argumente: Die Eingangsfunktion der Zeit und die Zeit bis zu welcher die Simulation berechnet werden soll.

```
let g = 7/((s+1)*(s+2)*(s+2.5));

plot(
  g.loop().sim(t->sin(2*t), 15).graph(),
)
```

Soll die Reaktion $g(t)$ auf einen Dirac-Impuls $\delta(t)$ berechnet werden, kann die Funktion `dirac([eps])` verwendet werden. Ihr Rückgabewert ist die Funktion

$$\delta_\varepsilon(t) \quad = \quad \begin{cases} \dfrac{1}{\varepsilon} & : \quad 0 \leq t < \varepsilon \\ 0 & : \quad \text{sonst} \end{cases}$$

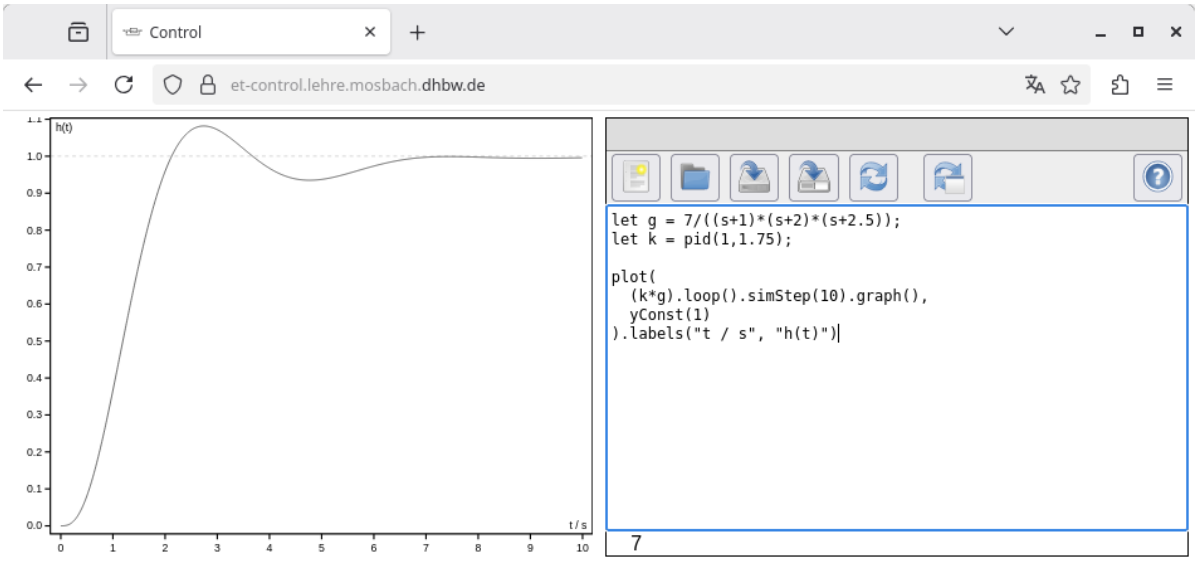


Abbildung 6: Systemreaktion im Zeitbereich

so dass gilt

$$\int_0^\infty \delta_\varepsilon(t) = \int_{-\infty}^\infty \delta_\varepsilon(t) \quad = \quad 1$$

Wird das Argument weggelassen, gilt $\varepsilon = 10^{-3}$ s.

```
let g = 7/((s+1)*(s+2)*(s+2.5));

plot(
  g.loop().sim(dirac(), 15).graph(),
)
```

Sowohl bei der Methode `simStep` als auch bei der Methode `sim` kann mit einem weiteren Argument die Schrittweite δt für die Berechnung nach dem Euler-Verfahren gesetzt werden. Der Vorgabewert ist $\delta t = t_{\text{Max}} \cdot 10^{-6}$.

4 Interaktivität

Um Zusammenhänge besser zu verstehen, kann es hilfreich sein, Abbildungen interaktiv zu gestalten. Dazu können Parameter, welche die Darstellung beeinflussen, über die Oberfläche einstellbar gemacht werden.

Dazu existiert die Konstante `gui`. Auf ihr sind drei Methoden deklariert:

slider(name, default, min, max) Erzeugt einen Schiebesteller (range slider). Angegeben werden ein Name - um bei mehreren Schiebestellern Verwechslungen zu ver-

hindern - ein Startwert und je ein Minimal- und Maximalwert. Die Rückgabe ist der eingestellte Wert, der dann mit dem Schiebesteller verändert werden kann.

```
let a = gui.slider("a", 0, -1, 1);
let b = gui.slider("b", 0, -1, 1);

{a:a, b:b}
```

Schiebesteller können z.B. genutzt werden, um Parameter eines linearen Systems wie Integrations- oder Vorhaltezeit veränderbar zu machen, um zu untersuchen, wie sich diese Änderung auf ein Bode-Diagramm, eine Wurzelortskurve oder die Systemreaktion $h(t)$ auswirkt.

check(name, default) Auch hier wird zunächst wieder ein Name angegeben und ein Wahrheitswert als Vorgabewert. Die Rückgabe ist dann der Wahrheitswert, der über ein Häkchen verändert werden kann.

```
let a = gui.check("a", false);

a
```

select(name, items...) Auch hier wird zunächst wieder ein Name angegeben und dan folgt eine Liste von Zeichenketten, aus denen einer Ausgewählt werden kann. Die erste Zeichenkette ist der Vorgabewert.

```
let a = gui.select("a", "Ja", "Nein", "Vielleicht");

a
```

Die interaktiven Elemente werden stets direkt unterhalb der eigentlichen Ausgabe angezeigt.

5 Listen

Listen sind der Datentyp, welcher über die meisten und flexibelsten Methoden verfügt. Im Folgenden seien die wichtigsten genannt. Weitere finden sich in der Online-Hilfe.

accept(func(item) bool) Diese Methode filtert eine Liste. Übergeben wird eine Funktion und diese Funktion wird für jedes Element der Liste ausgewertet. Ist das Ergebnis `true` wird das Element in die neue Liste übernommen, ist das Ergebnis `false` wird das Element nicht in die neue Liste übernommen.

```
[0,1,2,3,4].accept(x->x>1 & x<4)
```

ergibt

```
[2,3]
```

map(func(item) newItem) Diese Methode erzeugt eine neue Liste, dessen Elemente aus der alten Liste erzeugt werden, indem auf die alten Elemente die gegebene Funktion angewendet wird.

```
[0,1,2,3,4].map(x->x^2)
```

ergibt

```
[0,1,4,9,16]
```

reduce(func(item, item) item) Diese Methode erwartet eine Funktion mit zwei Argumenten und wendet diese Funktion auf je zwei Listenelement an, bis nur noch ein Element übrig ist. Dieses wird zurückgegeben.

```
["a","b","c","d"].reduce((a,b)->a+", "+b)
```

ergibt

```
"a, b, c, d"
```

sum() Diese Methode summiert alle Werte in einer Liste auf.

```
[1,4,7,12].sum()
```

ergibt 24. Es handelt sich um eine verkürzte Schreibweise von

```
[1,4,7,12].reduce((a,b)->a+b)
```

graph(func(item) x, func(item) y) Diese Methode hat zwei Funktionen als Argumente, die aus jeweils einem Element der Liste einen x- bzw. y-Wert erzeugen. Zurückgegeben wird der Graph dieser (x, y)-Paare.

```
plot(
  [1,2,3,4].graph(i->i,i->i^2)
)
```

Beide Funktionen können weggelassen werden. Dann müssen die Elemente der Liste entweder komplexe Zahlen, Vektoren oder selbst Listen mit je einem x- und y-Wert sein.

```
plot(
  [[1, 1],
   [2, 4],
   [3, 9],
   [4,16]].graph()
)
```